

Flutter Course

Section No.2

instructor
Youssef Hossam Elden

Table of contents

01

Control flow

(if, else if, else)

Switch Case

Loops (for, while, do-while)

03

Functions

Defining a Function

Calling a Function

Function Parameters

02

Advanced Control flow

Break and Continue

Iterating Collections (for-in)

Exception Handling (try, catch, finally)

04

Advanced Functions

Optional Parameters

Named Parameters

Returning Values

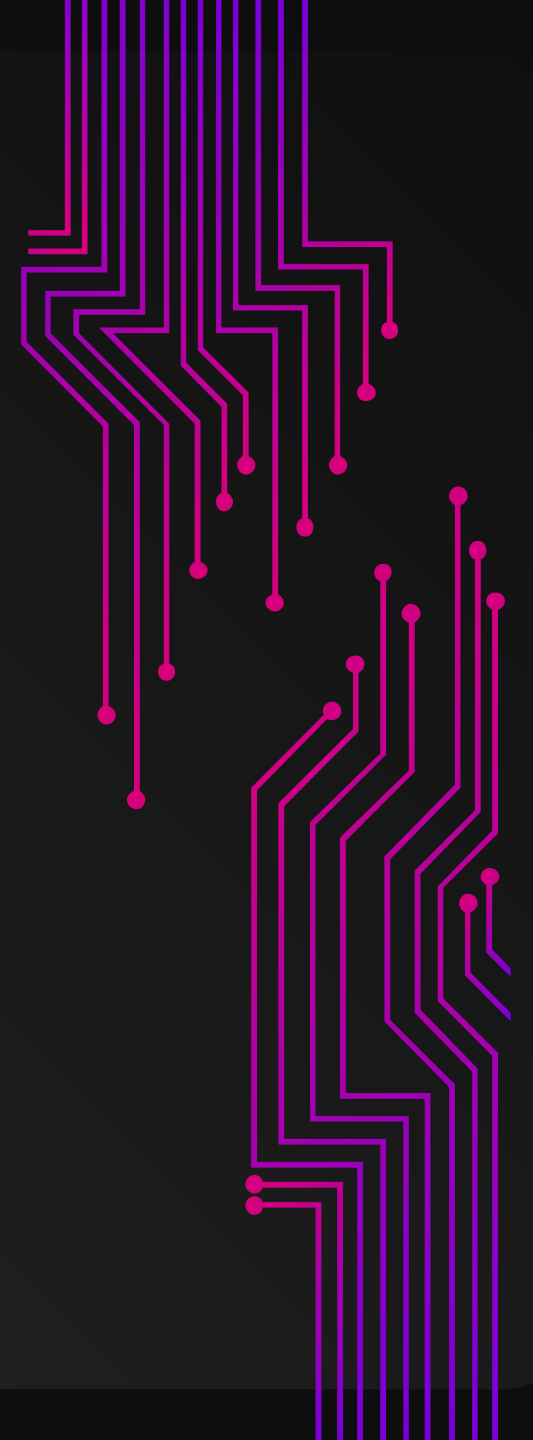
01

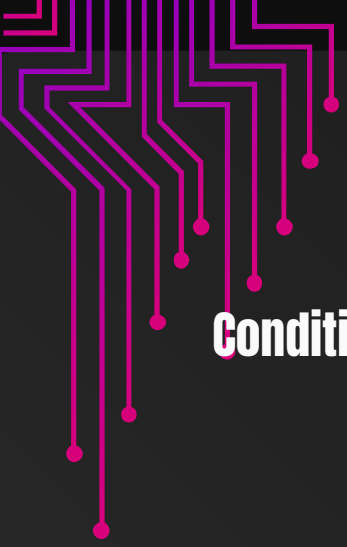
Control flow

(if, else if, else)

Switch Case

Loops (for, while, do-while)





IF statements

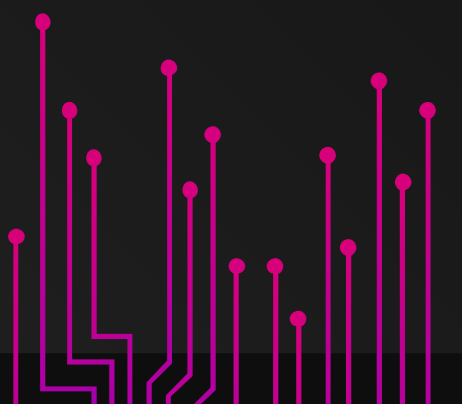
Conditional statements allow you to execute different blocks of code based on specified conditions.

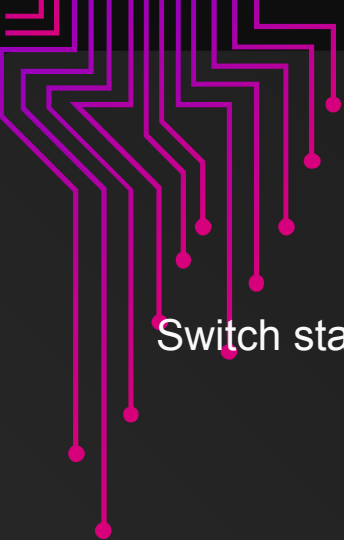
definition

```
if (isRaining()) {  
    you.bringRainCoat();  
} else if (isSnowing()) {  
    you.wearJacket();  
    } else {  
    car.putTopDown();}
```

using

```
int age = 25;  
if (age < 18) {  
    print("You are a minor.");  
} else if (age >= 18 && age < 60) {  
    print("You are an adult.");  
    } else {  
    print("You are a senior citizen.");  
    }
```





Switch Case

Switch statements allow you to select one of many code blocks to execute based on the value of an expression.

definition

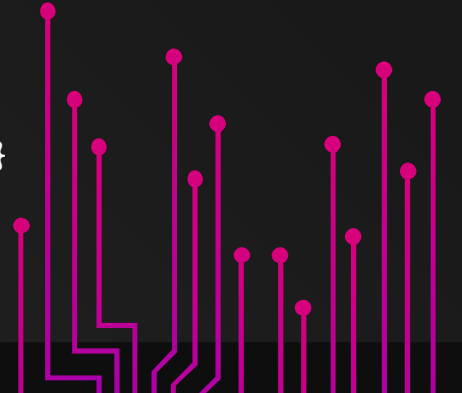
```
var command = 'OPEN';
switch (command) {
  case 'CLOSED': executeClosed();
  case 'PENDING': executePending();
  case 'APPROVED': executeApproved();
  case 'DENIED': executeDenied();
  case 'OPEN': executeOpen();
  default: executeUnknown();
}
```

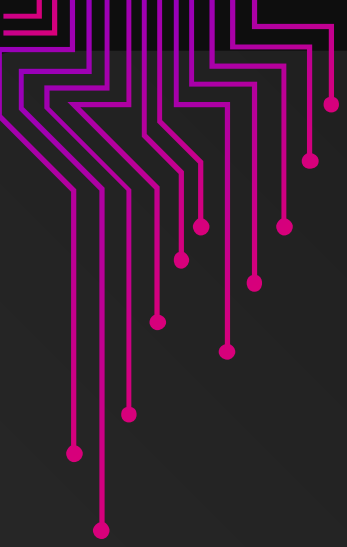
using

```
String day = 'Monday';
switch (day) { case 'Monday':
  print('It\'s the start of the week.');
```

```
    break;
  case 'Friday': print('It\'s almost the
    weekend!');
```

```
    break;
  default: print('It\'s a regular day.');
```





Loops (for, while, do-while)

Loops are used to repeat a block of code multiple times.

For Loop

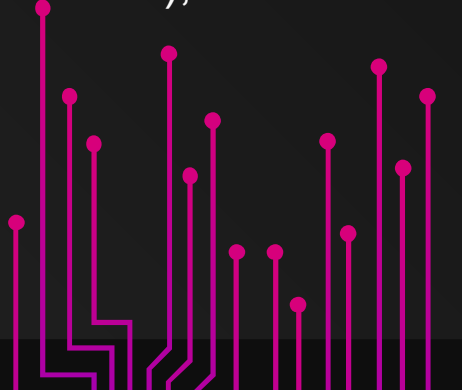
```
for (int i = 1; i <= 5; i++) {  
    print('Iteration $i');  
}
```

While Loop

```
int count = 0;  
while (count < 3) {  
    print('Count: $count');  
    count++;  
}
```

Do-While Loop

```
int num = 5; do {  
    print('Number: $num');  
    num--;  
}  
while (num > 0);
```



Practise Time

Exercise 1: Grade Calculator

Write a Dart program that prompts the user to enter their test score (0-100) and then uses if statements to determine their letter grade (A, B, C, D, or F). Display the letter grade as output.

Exercise 2: Leap Year Checker

Create a Dart program that checks if a given year is a leap year or not. Prompt the user to input a year and use an if statement to determine if it's a leap year. Leap years are divisible by 4, except for years divisible by 100 unless they are also divisible by 400.

Exercise 3: Sum of Numbers

Write a Dart program that calculates the sum of all numbers from 1 to a user-specified positive integer using a for loop. Prompt the user for the input and display the result.

Practise Time

Exercise 4: **Multiplication Table**

Create a Dart program that generates and displays the multiplication table (up to 10) for a user-specified number using a for loop. Prompt the user for the number.

Exercise 5: **Number Guessing Game**

Write a Dart program that generates a random number between 1 and 100. Then, let the user guess the number using a while loop. Provide feedback (too high, too low, or correct) and count the number of attempts it takes to guess correctly.

Exercise 6: **Factorial Calculator**

Create a Dart program that calculates the factorial of a positive integer entered by the user using a while loop. The factorial of a number is the product of all positive integers from 1 to that number.

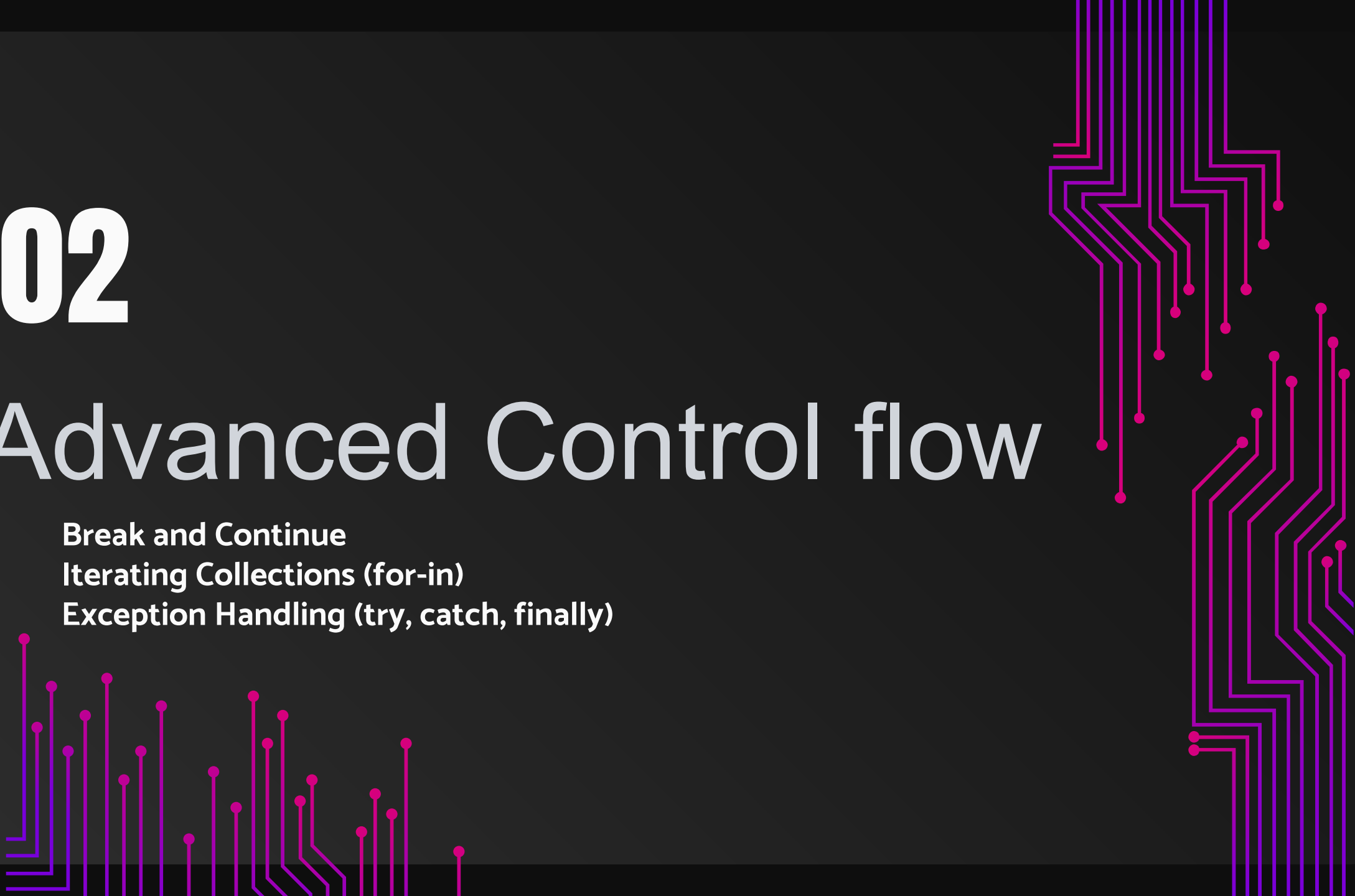
02

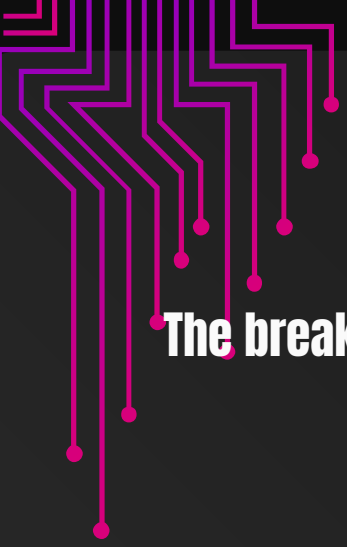
Advanced Control flow

Break and Continue

Iterating Collections (for-in)

Exception Handling (try, catch, finally)





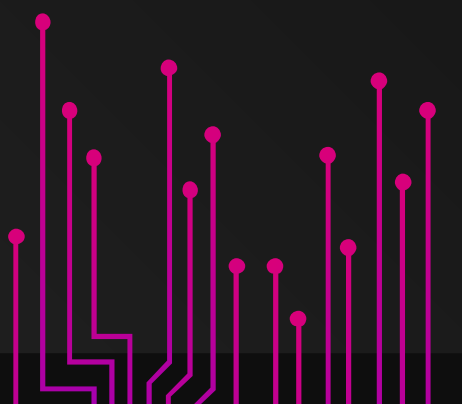
Break and Continue

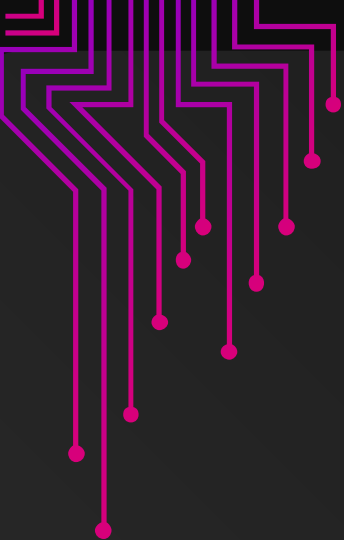
The break statement is used to exit a loop prematurely, while the continue statement is used to skip the current iteration and continue to the next one.

Example

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        break; // Exit the loop when i is 3  
    }  
    print('Iteration $i');  
}
```

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        continue; // Skip iteration when i is 3  
    }  
    print('Iteration $i');  
}
```





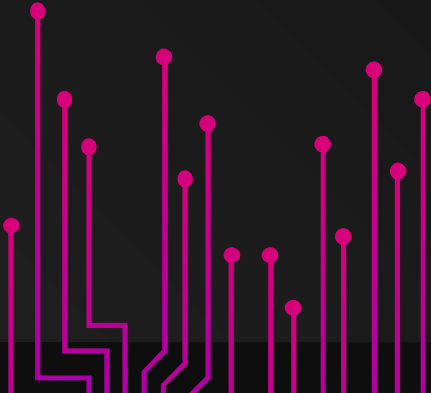
Iterating Collections (for-in)

The for-in loop is used to iterate over elements in a collection, such as lists or maps.

Example

```
List<int> numbers = [1, 2, 3, 4, 5];  
for (int number in numbers) {  
    print('Number: $number');  
}
```

```
Map<String, int> grades = {'Alice': 95, 'Bob': 87, 'Charlie': 92};  
for (String name in grades.keys) {  
    print('$name\'s grade: ${grades[name]}');  
}
```





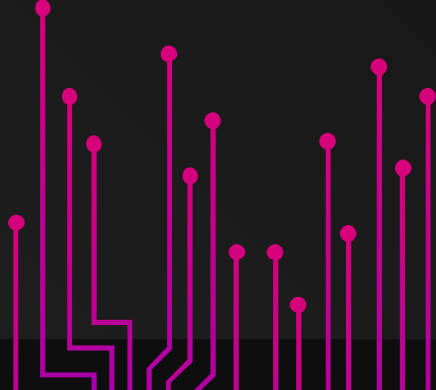
Exception Handling (try, catch, finally)

Dart provides exception handling mechanisms to gracefully handle errors and exceptions.

Example

```
try {  
  int result = 10 ~/ 0; // Division by zero  
  print('Result: $result');  
} catch (e) {  
  print('An error occurred: $e');  
} finally {  
  print('Execution completed.');
```

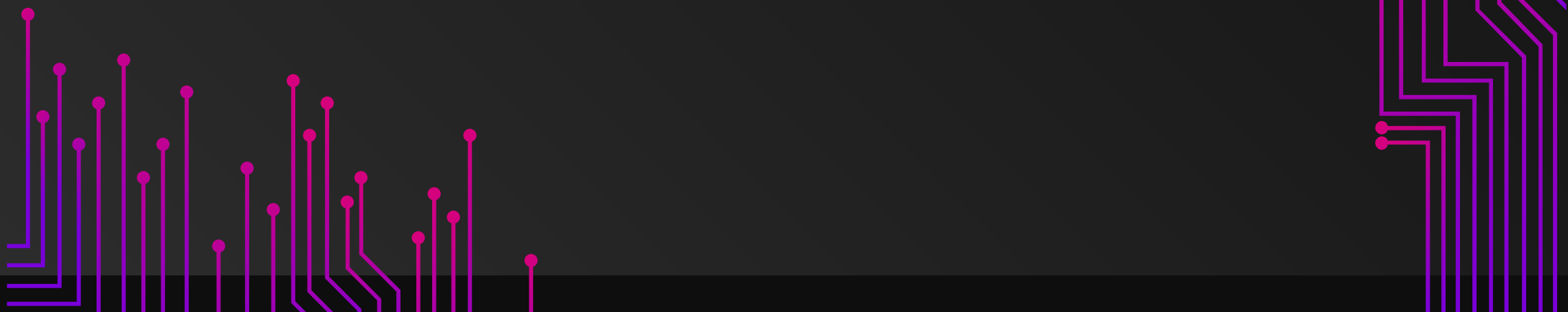
```
}
```



03

Functions

Defining a Function
Calling a Function
Function Parameters





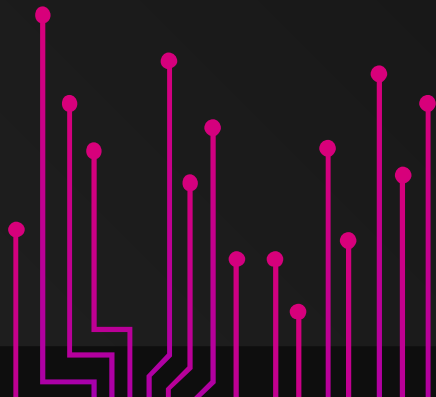
Defining a Function

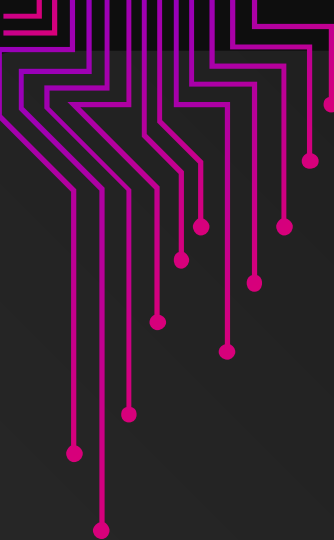
You can define a function in Dart using the void keyword (if it doesn't return a value) or by specifying a return type (if it does return a value).

Example

```
void sayHello() {  
  print('Hello, World!');  
}
```

```
int add(int a, int b) {  
  return a + b;  
}
```



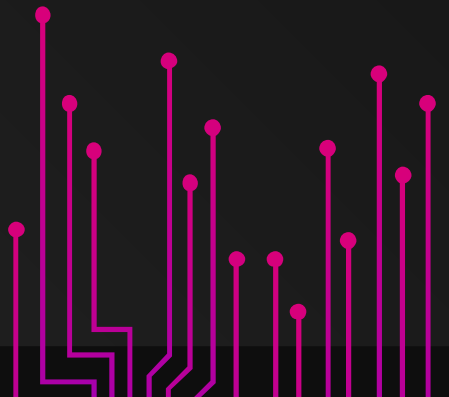


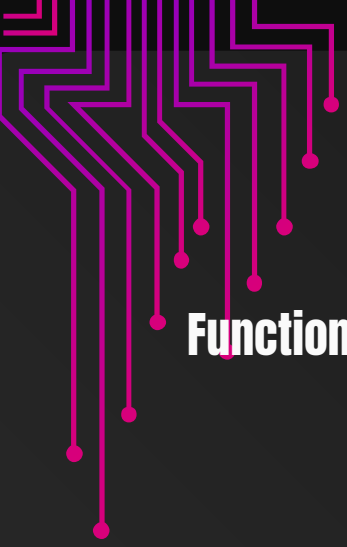
Calling a Function

To execute a function and perform the tasks defined within it, you need to call it.

Example

```
sayHello(); // Calling the sayHello function  
int sum = add(5, 3); // Calling the add //function  
and storing the result in 'sum'  
print('The sum is $sum'); // Printing the result
```





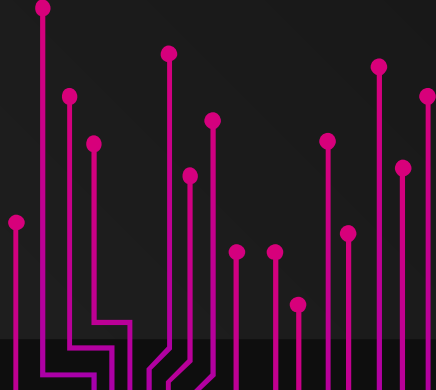
Function Parameters

Functions in Dart can have zero or more parameters. Parameters are like placeholders for values that you pass to the function when calling it..

Example

```
void greet(String name) {  
    print('Hello, $name!');  
}
```

```
greet('Alice'); // Calling the greet function //with  
                the 'name' parameter
```



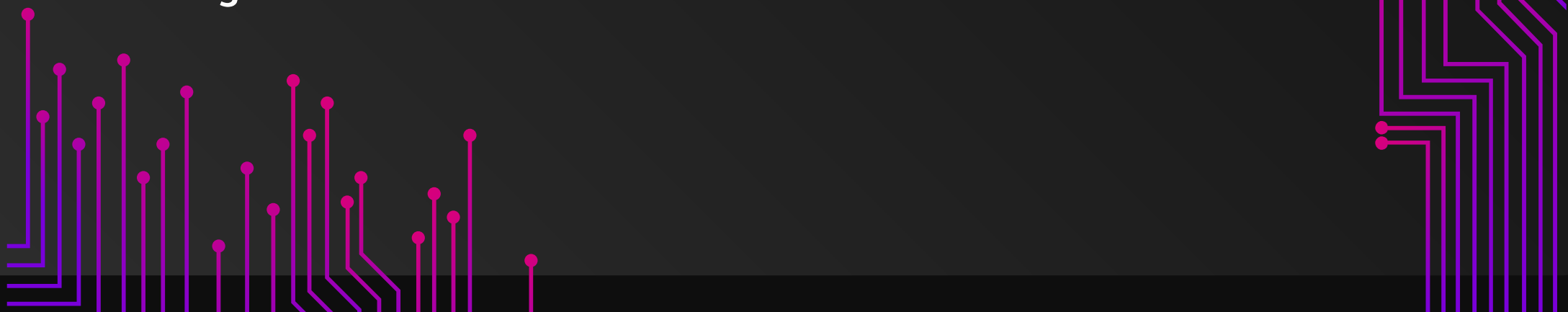
04

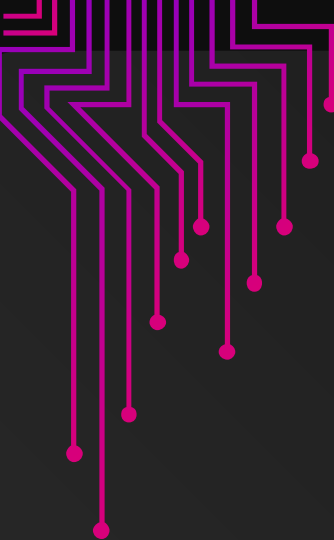
Advanced Functions

Optional Parameters

Named Parameters

Returning Values





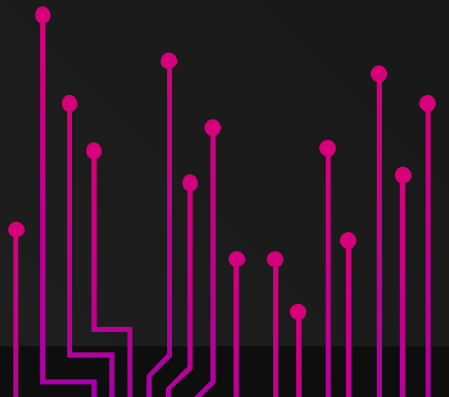
Optional Parameters

You can make function parameters optional by enclosing them in square brackets [].

Example

```
void greetWithOptional([String? name]) {  
    if (name != null) {  
        print('Hello, $name!');  
    } else {  
        print('Hello, World!');  
    }  
}
```

```
greetWithOptional(); // Outputs: Hello, World!  
greetWithOptional('Bob'); // Outputs: Hello, Bob!
```





Named Parameters

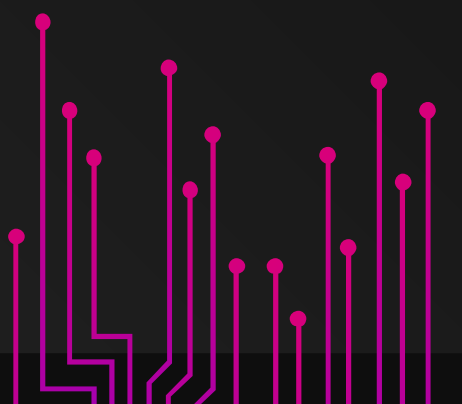
Dart also supports named parameters, which allow you to specify parameters by name when calling a function, making the code more readable and flexible.

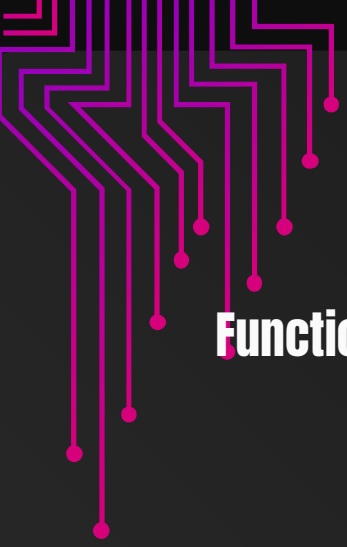
Example

```
void greetWithNamed({String? firstName, String? lastName}) {  
    if (firstName != null && lastName != null) {  
        print('Hello, $firstName $lastName!');  
    } else {  
        print('Hello, World!');  
    }  
}
```

```
greetWithNamed(firstName: 'John', lastName: 'Doe'); // Outputs:  
Hello, John Doe!
```

Here, you can specify firstName and lastName as named parameters when calling greetWithNamed.





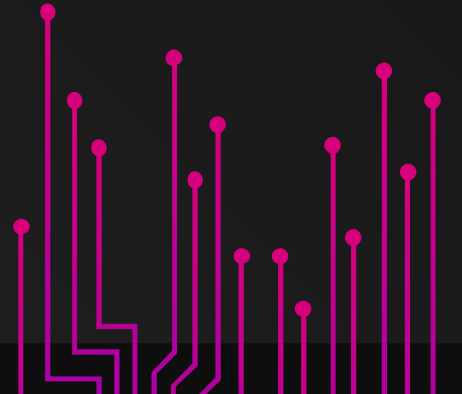
Returning Values

Functions can return values using the return statement. The return type should match the declared return type of the function.

Example

```
int add(int a, int b) {  
    return a + b;  
}
```

In this example, add returns an int value, which is the sum of a and b.



Practise Time

Exercise 1: Greeting Function

Create a Dart program that defines a function `greet` which takes a person's name as a parameter and prints a greeting message. Prompt the user to enter their name and then call the `greet` function to greet them.

Exercise 2: Calculator Function

Write a Dart program that defines a function `calculate` to perform basic arithmetic operations (addition, subtraction, multiplication, division) on two numbers provided as parameters. Prompt the user to enter two numbers and the operation they want to perform, then call the `calculate` function to display the result.

Practise Time

Exercise 3: Temperature Converter

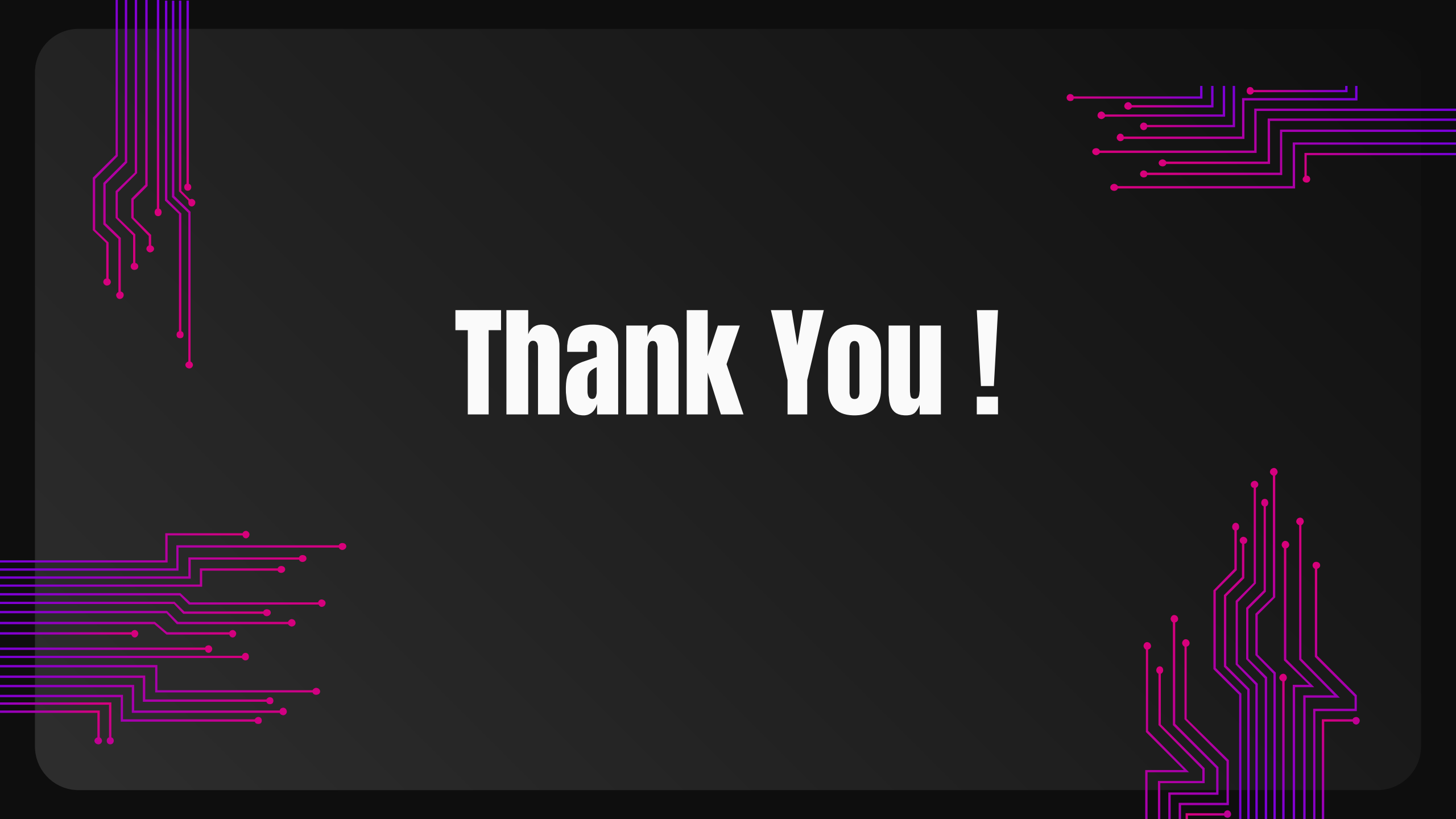
Build a Dart program that defines two functions, `celsiusToFahrenheit` and `fahrenheitToCelsius`, to convert temperatures between Celsius and Fahrenheit. Prompt the user for a temperature value and a choice of conversion using function parameters, and then call the appropriate function to display the converted temperature.

Exercise 4: Area Calculator

Create a Dart program that defines a function `calculateArea` for calculating the area of different shapes (e.g., square, rectangle, circle) based on the user's choice and provided parameters. Prompt the user to choose a shape, enter the necessary values (e.g., side length, length, width, radius), and call the `calculateArea` function to display the area.

The image features a dark gray background with a central white text element. In the four corners, there are decorative graphic elements consisting of thin, light blue lines that branch out and end in small dots, resembling a stylized circuit board or data connections. The lines are more dense in the top-left and bottom-right corners and more sparse in the top-right and bottom-left corners.

Questions !

The image features a dark gray background with a central white text area. In the four corners, there are decorative elements resembling circuit board traces. These traces are drawn in a light blue color and end in small blue dots. The traces in the top-left and bottom-left corners extend from the left edge towards the center. The traces in the top-right and bottom-right corners extend from the right edge towards the center. The overall aesthetic is clean and modern, with a technological or digital theme.

Thank You !